Lesson 7 Teacher's Key: On the Move

TIC-80 - Instructor Reference Only

VIDEO GAME CREATION

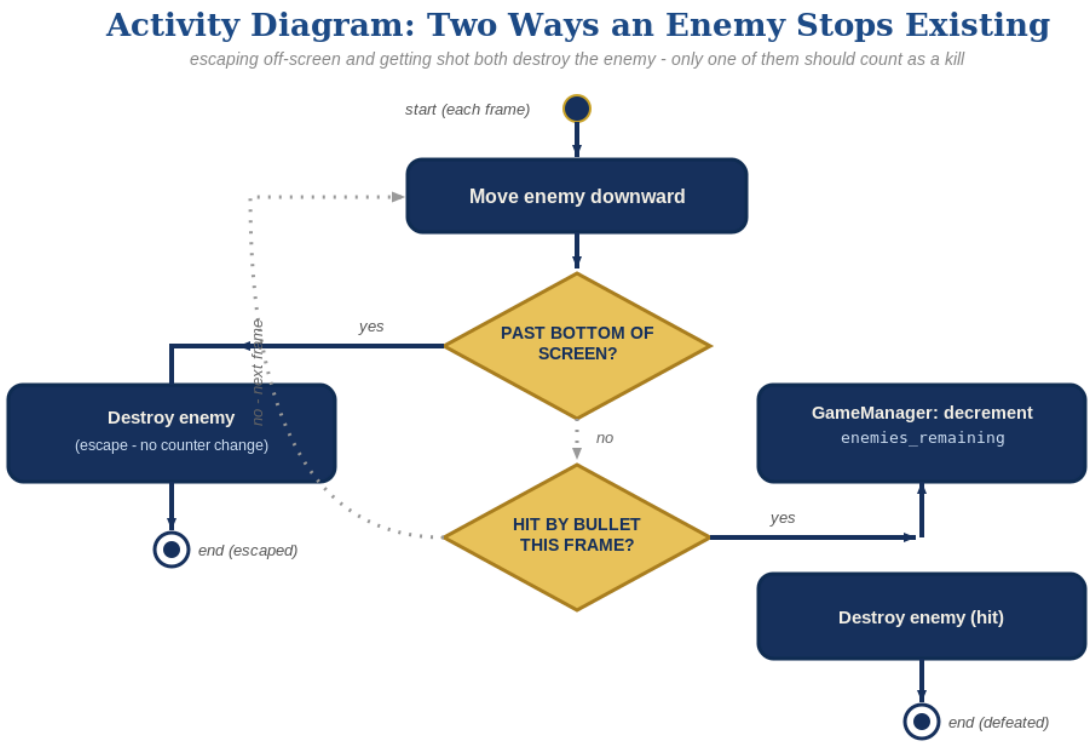
What This Lesson Teaches

The same escape-vs-hit distinction expressed as two clearly separate loops over the same enemies table - one that only ever sets enemy.alive=false for movement/escape reasons, and a separate one (already existing since Lesson 4) that both sets enemy.alive=false AND decrements enemies_remaining for an actual hit. This lesson also retrofits the drawing loop's variable names (j/e) to enemy_index/enemy, matching the meaningful-name standard the rest of the course already follows.

The Bug Being Fixed

No bugfix needed for TIC-80 - Lesson 6's win-counter decrement already lived only inside the bullets-vs-enemies collision loop, and this lesson's new movement/escape logic is added to the separate enemy-drawing loop, which never touches enemies_remaining. This lesson does rename the drawing loop's j/e variables to enemy_index/enemy while it's already being edited - a small Rule 11 cleanup carried over from earlier lessons, not a functional change.

Level 1 - Full Worked Solution



These two checks look similar - both end with "the enemy is gone" - but only the hit check is allowed to touch enemies_remaining. Attaching the counter to a generic "was destroyed" hook instead of this specific hit check is exactly the bug GameMaker and 001 Game Creator have to fix this lesson.

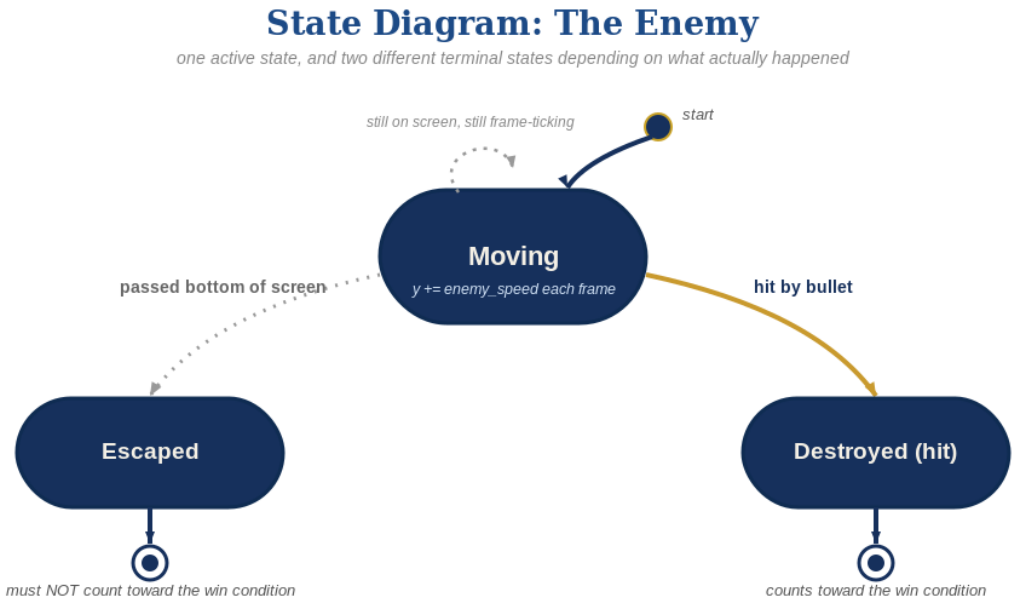
The general escape-vs-hit distinction behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

- Movement (enemy.ey=enemy.ey+0.3) and the escape check (enemy.ey>136) both live in the existing enemy-drawing loop at the bottom of TIC(), guarded by if enemy.alive then, same structural location as before.
- enemy.alive=false in the escape branch removes the enemy from being drawn or collision-checked on future frames, with zero reference to enemies_remaining.
- The bullets-vs-enemies collision loop (from Lesson 4/6) is untouched by this lesson - it remains the only place enemies_remaining changes.

```
-- enemy movement/escape (drawing loop):
for enemy_index=1,#enemies do
 local enemy=enemies[enemy_index]
 if enemy.alive then
  enemy.ey=enemy.ey+0.3
  if enemy.ey>136 then
   enemy.alive=false
  else
   spr(3,enemy.ex,enemy.ey,0)
  end
 end
end
```

Why This Structure

- TIC-80's table-driven design already forces students to be explicit about which loop does what, since there's no automatic event system doing it for them - this lesson leans on that explicitness rather than fighting it.
- Keeping the two loops textually separate (rather than merging movement/escape into the collision loop 'to save a pass over the table') keeps each loop's single job legible, even though it costs a small amount of performance running two loops instead of one - worth naming that tradeoff explicitly for older/more advanced students.
- The rename from j/e to enemy_index/enemy is purely cosmetic (Lua doesn't care what a local is called), but it matches the meaningful-name standard already applied to every other engine's loop variables since Lesson 5 - worth a one-sentence mention in class if a sharp-eyed student compares this week's code to a screenshot of an earlier lesson.



Once an enemy leaves Moving, it never comes back - both destinations are final. The whole trick this lesson teaches is making sure your code can tell these two final states apart, since only one of them is a kill.

The enemy's two possible endings this week - only one of them should count as a kill.

Level 2 / Level 3 Talking Points

- For advanced students: ask whether the two loops could be merged into one pass over the enemies table, and what would be gained (a few percent of performance) versus lost (harder-to-follow code, higher risk of accidentally mixing the two concerns) - this is a genuine, defensible design tradeoff, not a right/wrong question.
- Point out that enemy.alive is the single shared piece of state both loops respect - it's what lets two independently-written loops cooperate safely without either needing to know about the other's internals.

Common Student Mistakes

- Adding the enemies_remaining decrement into the movement/escape branch by mistake, copy-pasting from the collision loop.

- Forgetting the if enemy.alive then guard on the movement loop, causing already-dead (but not yet cleared) table entries to keep moving/drawing.
- Using the wrong screen-height constant (TIC-80's default is 136 pixels tall, not 128 or 240 - easy to misremember).